CIDADES ESG INTELIGENTES

Relatório técnico de DevOps
Carbono API — gestão de emissões e sustentabilidade

Disciplina	DevOps
Equipe	Preencher com os nomes dos integrantes
Data	Preencher na submissão

Nota de integridade: os arquivos de automação e a documentação foram preparados, mas o pipeline não foi executado com sucesso neste ambiente e os ambientes de staging/produção não foram implantados. A documentação identifica claramente essas dependências e não apresenta evidências fictícias.

1. Objetivo

Preparar a API existente para um fluxo DevOps reproduzível, reduzindo erros manuais entre alteração de código, testes, empacotamento e implantação. A solução mantém a implementação Java/Spring Boot recebida e acrescenta automação, containerização e orientações operacionais.

2. Tecnologias e arquitetura

Java 21, Spring Boot, Maven, Oracle Database Free, Flyway, Spring Security/JWT, JUnit 5, H2 para o perfil de teste, Docker, Docker Compose, GitHub Actions e GitHub Container Registry (GHCR).

A aplicação está organizada em controllers, services, repositories, entities, DTOs e configuração de segurança. O banco utiliza migrações Flyway. A coleção Insomnia original foi incluída no pacote para validação manual dos endpoints.

3. Pipeline CI/CD

1) Checkout do commit. 2) Configuração de Java 21 e cache Maven. 3) Execução de **mvn -B clean verify**. 4) Publicação do relatório Surefire. 5) Build e publicação da imagem Docker no GHCR, com tags do SHA e latest. 6) Deploy em staging após push na branch main. 7) Deploy em produção manual, por workflow_dispatch e com proteção do ambiente GitHub.

O job de imagem depende do sucesso do build/teste. Pull requests executam validação, mas não publicam a imagem. O deploy real exige secrets de SSH e hosts configurados pela equipe.

4. Containerização

O Dockerfile usa duas etapas. A etapa Maven/Temurin 21 executa clean verify; a etapa final utiliza somente JRE, copia o JAR e executa como usuário não-root. O Compose local separa API e Oracle em uma rede bridge. O volume oracle_data preserva os dados do banco.

As credenciais são fornecidas por variáveis de ambiente. O Compose de deploy consome uma imagem versionada do GHCR e não publica a porta do Oracle externamente. Para exposição pública, recomenda-se proxy reverso com TLS, firewall, backups e monitoramento.

5. Testes e qualidade

O pipeline executa Maven Verify e guarda relatórios Surefire. O teste de contexto foi isolado no perfil test, com H2 em memória e Flyway desabilitado, para não exigir Oracle durante o teste inicial. Isso não substitui testes unitários adicionais nem testes de integração contra Oracle.

A execução de ./mvnw -B clean verify foi tentada durante a preparação, mas o Maven Wrapper não conseguiu baixar a distribuição Maven do Maven Central neste ambiente. Por isso, não afirmamos que os testes passaram; é necessário repetir a execução em ambiente com acesso ao repositório de dependências.

6. Segurança

Foram removidas credenciais fixas da configuração principal e do Compose. O pacote inclui .env.example com marcadores, não um .env real. O .gitignore impede versionamento acidental do .env. Valores que já tenham sido compartilhados devem ser rotacionados.

No GitHub, os segredos de SSH e caminhos de deploy devem ser armazenados em secrets por ambiente. Use segredos diferentes para staging e produção, restrinja o acesso ao banco e não inclua tokens em capturas de tela.

7. Desafios e soluções

Credenciais fixas: variáveis de ambiente e rotação. Build que ignorava testes: Dockerfile executa clean verify. Teste dependente do Oracle: perfil test com H2. Falta de automação: workflow GitHub Actions.

Diferença entre ambientes: Compose de deploy e secrets separados. Falta de evidências: roteiro de captura após executar o pipeline e os ambientes reais.

8. Evidências a anexar após execução

Captura do workflow com build/testes aprovados; imagem publicada no GHCR com SHA; saída de docker compose ps em staging; teste funcional da API em staging; aprovação e resultado do deploy de produção; requisição Insomnia bem-sucedida com dados sensíveis ocultos. Não foram inseridas capturas fictícias.

9. Checklist de entrega

[x] Código-fonte no ZIP; [x] Dockerfile; [x] Docker Compose com volume, variáveis e rede; [x] workflow com build/teste/publicação/deploy; [x] README; [x] documentação técnica; [x] .env.example; [x] coleção Insomnia.

[ ] Pipeline executado com sucesso no GitHub — depende da publicação do repositório. [ ] Staging implantado e verificado — depende de host/secrets. [ ] Produção implantada e verificada — depende de host/secrets/aprovação. [ ] Capturas reais anexadas após execução.

10. Conclusão

A aplicação foi preparada para adoção de práticas DevOps, com build multi-stage, testes como gate, publicação de imagem e desenho de implantação por ambientes. A entrega de código e documentação está preparada para revisão; a conclusão operacional depende de executar o pipeline e comprovar staging e produção com infraestrutura real.

Checklist operacional do servidor

Ambiente	Configuração necessária
Staging	STAGING_HOST, STAGING_USER, STAGING_SSH_KEY, STAGING_PATH e arquivo .env no host.
Produção	PRODUCTION_HOST, PRODUCTION_USER, PRODUCTION_SSH_KEY, PRODUCTION_PATH, arquivo .env e aprovação do ambiente GitHub.